SolutionArchitectAI. Ключевые технические решения

Технические решения

Цель документа — зафиксировать планируемые технические решения по реализации MVP системы SolutionArchitectAI для согласования с системными архитекторами, техническими руководителями, Python- и frontend-разработчиками.

Содержание

1. Назначение и общая архитектура 2

2. Backend 3

3. Хранение данных 3

4. Искусственный интеллект 4

5. Пользовательский интерфейс 5

6. Инструменты качества 5

---

1. Назначение и общая архитектура

SolutionArchitectAI помогает архитектору перейти от требований и корпоративного контекста к согласованной архитектуре. Рабочее место объединяет общую модель объектов и связей, её диаграммы, свойства, документы, чат с ИИ и историю решений. Предложения ИИ проходят проверку; решение об их принятии остаётся за человеком.

Frontend и backend планируется реализовать как отдельные части приложения. Frontend на **React и TypeScript** отвечает за рабочее место архитектора. Backend на **Python 3.13 и FastAPI** предоставляет HTTP API, выполняет агентный процесс, проверяет изменения и обращается к хранилищам. Браузер работает через API и не получает прямого доступа к базе данных, Git или серверу LLM.

Локальный запуск и дальнейшее развёртывание

Для локального запуска разработчиком планируется использовать **Docker Compose**. Frontend, backend и PostgreSQL запускаются в отдельных контейнерах. Отдельный контейнер **PlantUML Server** генерирует схемы внутри локального окружения. Для Git-репозиториев моделей и загруженных файлов предусматриваются сохраняемые данные. Сервер LLM подключается отдельно и не обязан входить в основной Compose.

Для будущего production планируется **Red Hat OpenShift**, где будут запускаться контейнерные образы компонентов. Compose остаётся средством локального запуска. Образы должны работать без привилегий, с конфигурацией и секретами платформы, проверками готовности и отдельным выполнением миграций. Размещение сохраняемых данных, особенно Git-репозиториев при масштабировании, требует отдельного эксплуатационного решения.

Единая модель и представления

Архитектурные объекты имеют устойчивые идентификаторы. Один объект может присутствовать на нескольких диаграммах: его свойства общие, а расположение элементов принадлежит конкретному представлению. Удаление элемента с диаграммы не означает удаления объекта из модели.

Интерактивное редактирование и генерация схем решают разные задачи. React Flow обслуживает работу с диаграммой в браузере. PlantUML воспроизводимо строит публикуемые схемы из проверенной модели; исходники не отправляются в публичные сервисы. Будущий обмен с Archi планируется через предметную модель и ArchiMate Model Exchange XML, независимо от внутреннего формата React Flow.

2. Backend

Планируется использовать следующий стек:

- **Python 3.13** — язык и целевая версия backend; **uv** управляет зависимостями, окружениями и lock-файлом для воспроизводимой установки.
- **FastAPI** — HTTP API для frontend, включая получение состояния длительных операций. Типизированные контракты и OpenAPI упрощают согласование API.
- **Dishka** — контейнер внедрения зависимостей: связывает сервисы с адаптерами хранения и LLM, управляет областями их использования.
- **Pydantic** — типизированные модели входных и выходных данных и их валидация; **Pydantic Settings** загружает и проверяет конфигурацию приложения.
- **SQLAlchemy** — работа с PostgreSQL и транзакциями; **Alembic** — управляемые миграции схемы БД. Это разделяет обычные операции с данными и изменение структуры базы.

Предметная модель архитектуры реализуется в Python и определяет объекты, отношения, разрешённые изменения и ограничения. **JSON используется для сериализации и восстановления модели**, сохраняемой в Git. JSON Schema проверяет формат данных, но не заменяет предметную логику.

Поддержка Python 3.13 проверена по метаданным основных пакетов и разрешению зависимостей для этой версии. Это не подтверждает поведение готового приложения; точные версии и интеграционные проверки закрепляются при реализации.

3. Хранение данных

PostgreSQL хранит пользователей, проекты, задачи, чаты, сообщения, запуски агентов, состояние выполнения, согласования и журнал операций. Здесь же сохраняются контрольные точки LangGraph, необходимые для продолжения агентного процесса.

Git хранит историю редакций общей архитектурной модели: изменения объектов и связей, архитектурные решения и связанные представления. Рабочие ветки задач отделяют предложения от согласованной редакции; принятие изменения, архитектурное согласование и merge выполняются как отдельные действия.

Локальное файловое или S3-совместимое хранилище содержит загруженные документы, вложения и крупные файлы, необходимые для воспроизведения архитектурных решений. Версии источников связываются с результатами проектирования.

Эти хранилища нужны одновременно. Git даёт сравнение редакций и ветвление архитектуры, PostgreSQL — транзакции и запросы к текущему состоянию совместной работы, файловое хранилище — хранение крупных вложений. Один Git неудобен для сообщений и конкурентного операционного состояния. Одна PostgreSQL потребовала бы заново реализовать свойства Git; помещение крупных файлов в архитектурный репозиторий увеличивает его объём и затрудняет сопровождение.

4. Искусственный интеллект

LangChain планируется использовать для взаимодействия с языковыми моделями. Базовые интерфейсы предоставляет `langchain-core`, а `langchain-openai` подключает настраиваемый OpenAI-совместимый сервер. Полный пакет LangChain не требуется для этого набора задач.

Единый компонент структурированной генерации на основе **LangChain и Pydantic** преобразует запросы в типизированные ответы и проверяет их структуру. Поддержка структурированного вывода зависит от модели и сервера и проверяется отдельно. Валидный ответ ещё не означает корректного архитектурного решения: изменения проходят предметные проверки и рассмотрение человеком.

LangGraph выполняет многоэтапный агентный процесс: извлечение требований, уточнение, проектирование и независимое ревью. Он выбран вместо собственного универсального движка выполнения, поскольку поддерживает сохранение состояния, прерывание для ответа человека и возобновление работы. **PostgreSQL checkpointer** сохраняет контрольные точки процесса; он не заменяет архитектурную модель в Git или журнал предметных операций.

Роли агентов

- **Агент извлечения требований** выделяет требования, противоречия и пробелы, сохраняя связь с источниками.
- **Агент уточнений** формирует вопросы архитектору и учитывает полученные ответы.
- **Агент проектирования** предлагает объекты, связи, решения и представления с объяснением оснований.
- **Агент архитектурного ревью** независимо проверяет кандидата и сообщает о нарушениях и сомнительных предложениях.

Агенты работают внутри backend. Они не согласовывают собственные предложения и не выполняют merge. Изменения общих объектов показываются вместе с влиянием на связанные представления, чтобы архитектор мог принять, отклонить или вернуть предложение на уточнение.

Локальные языковые модели

LM Studio планируется использовать как удобный сервер локальной модели для разработки, **vLLM** — как вариант выделенного сервера inference. Оба подключаются отдельно через настроенный endpoint. Совпадение HTTP-формата не гарантирует одинаковых возможностей: конкретные пары модель/сервер требуют проверки структурированных ответов, потоковой выдачи и отмены. Автоматическая отправка данных внешнему провайдеру не предусматривается.

5. Пользовательский интерфейс

React формирует интерактивное рабочее место из компонентов; **TypeScript** проверяет типы данных и контрактов на стороне frontend. **Vite** обеспечивает локальную разработку и сборку SPA. Серверный рендеринг для MVP не требуется.

React Router отвечает за маршруты пространства, задачи, вида и редакции. **TanStack Query** загружает, кеширует и обновляет серверные данные FastAPI. Маршрутизация отделена от кеша; локальные несохранённые правки не подменяют подтверждённое состояние сервера.

shadcn/ui + Tailwind CSS выбраны для форм, таблиц, диалогов и общей темы. Компоненты доступны для изменения в проекте и сочетаются со стилизацией чата; команда сопровождает скопированный код. Обязательны светлая, тёмная и системная темы, читаемые русские подписи и управление с клавиатуры.

assistant-ui выбран для чата с ИИ: предоставляет компоненты сообщений и механизм работы с потоковыми ответами. Собственный адаптер соединяет его с FastAPI; история, вложения и состояние запусков сохраняются на сервере. Карточки вопросов и предложений реализуются под предметные команды приложения; облачный сервис чата не требуется.

React Flow (`@xyflow/react`) выбран для интерактивных архитектурных диаграмм: выбор и перемещение элементов, работа со связями, масштабирование и собственные типы узлов. Приложение реализует панели свойств, правила отношений, независимое расположение элементов по видам и diff предложений ИИ с принятием или отклонением. Готовый SVG PlantUML пригоден для просмотра и публикации, но этих операций редактирования не предоставляет.

6. Инструменты качества

Backend:

- **Ruff** — линтинг и форматирование Python; **Pyright** — статическая проверка типов.
- **import-linter** — контроль разрешённых зависимостей между пакетами.
- **pytest** — автоматизированные тесты предметных правил и интеграций.

Frontend:

- **ESLint** — статический анализ TypeScript/React; **Prettier** — форматирование.
- **TypeScript strict** — строгая проверка типов; **Vitest** — модульные и компонентные тесты.
- **Playwright** — сквозные проверки работы приложения в браузере.